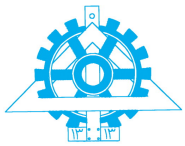


اندازه گیری اینترنت

نام و نام خانوادگی: سید علی آقامیری



اتمرین۳

الف) $Jaeger$ یک سیستم مانیتورینگ و تراسینگ متن‌باز است که توسط شرکت $Uber$ توسعه داده شده است. این سیستم به تیم‌های توسعه نرم‌افزار کمک می‌کند تا عملکرد و عملیات سیستم‌های خود را مانیتور کرده و مشکلات را پیگیری کنند.$Jaeger$ از تکنولوژی تراسینگ استفاده می‌کند تا بتواند فرآیندهایی که در سیستم اجرا می‌شوند را دنبال کرده و زمان اجرا، خطاها، وابستگی‌ها و سایر اطلاعات مربوط به آنها را ثبت کند.در واقع با این نرم افزار میتوانیم مسیر عبور درخواست ها / فراخوانی های داخل توابع، کلاس ها و حتی سرویس های مختلفمان را با آن رصد کنیم. همچنین این نرم افزار زمان دقیق اجرای هر کدام از پردازش ها ازجمله پردازش های فرزند را نشان میدهد که میتواند در بهینه کردن زمان پردازش سیستم کمک کننده باشد، این اطلاعات به تیم‌های توسعه نرم‌افزار کمک می‌کند تا مشکلات و عیب‌های سیستم را شناسایی و رفع کنند.

دو جزء اصلی این سیستم عبارت است از:

$Span$: هر اتفاقی از جمله یک درخواست $http$، فراخوانی یک تابع یا اجرای یک کوئری دیتابیس یک $span$ محسوب میشه که کوچکترین واحد کاری $Jaeger$ هست. یک $span$ از یک ایدی ویژه، نام، زمان شروع و طول زمان اجرا تشکیل شده.

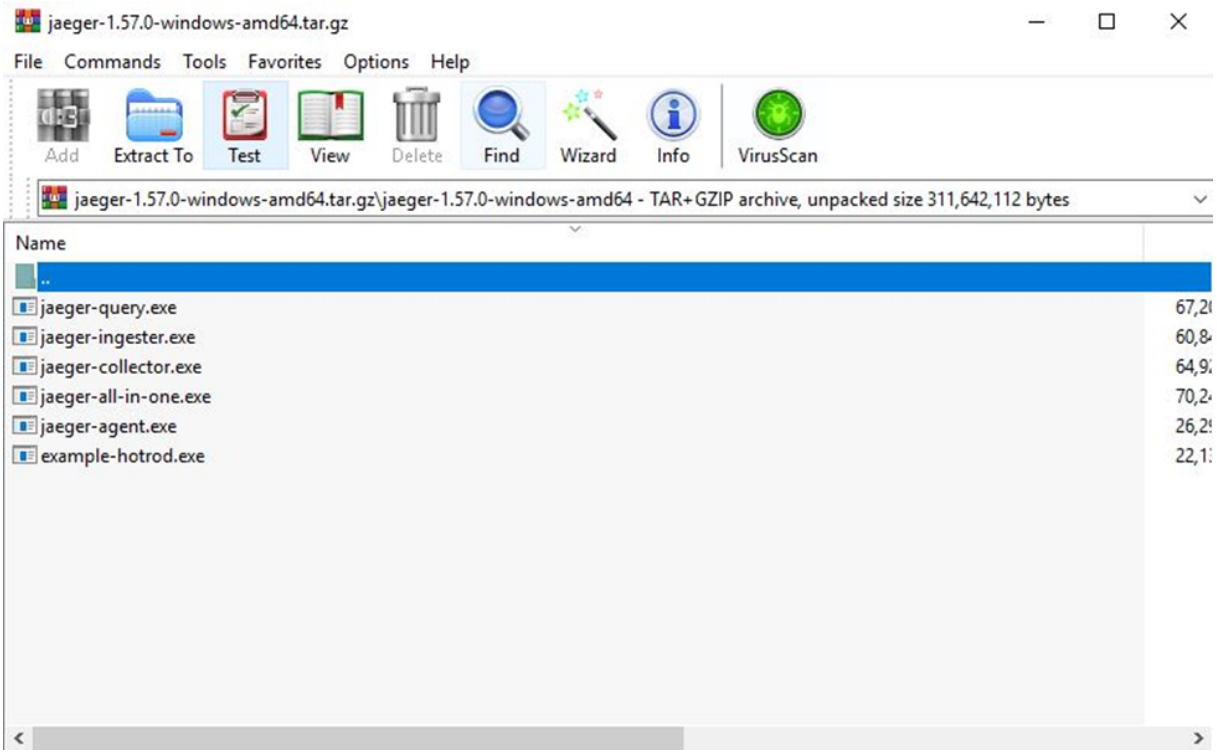
$Trace$: یک مجموعه از $span$های متصل به هم است که رابطه پدر/فرزندی با هم دارند.

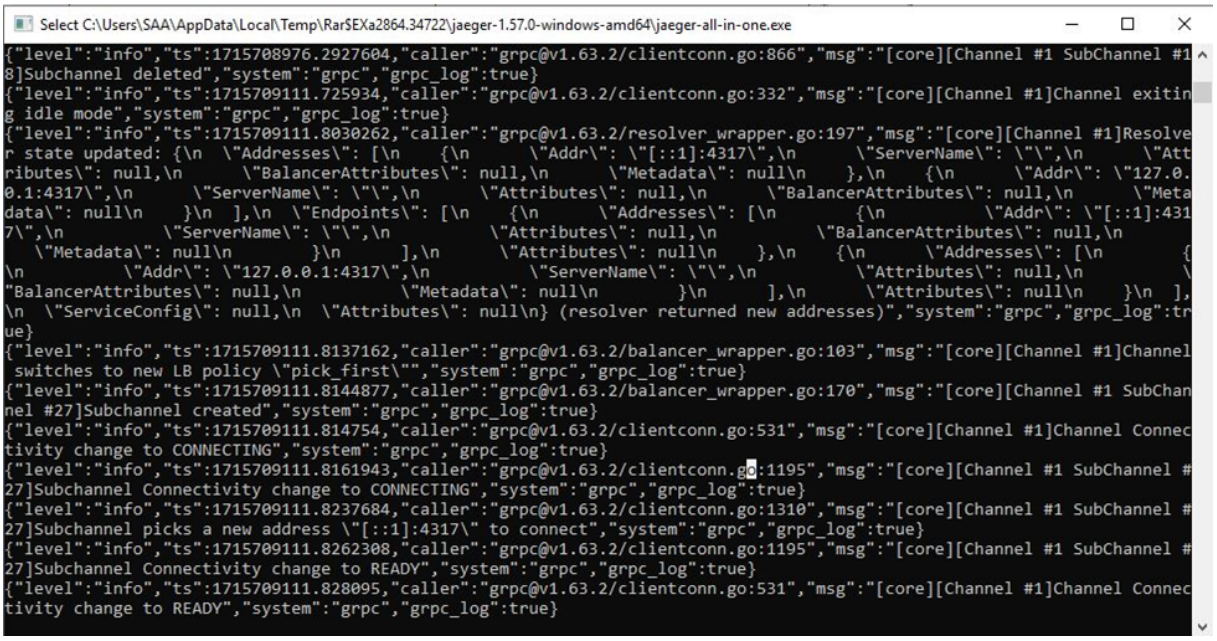
$Jaeger$ به عنوان یک سیستم تراسینگ و مانیتورینگ، کاربردهای متعددی دارد که شامل موارد زیر می‌شود: ۱. ردیابی و پیگیری عملکرد: $Jaeger$ به توسعه‌دهندگان کمک می‌کند تا فرآیندهای مختلفی که در سیستم اجرا می‌شوند را ردیابی و پیگیری کنند. این امکان به آنها می‌دهد تا زمان اجرا، وابستگی‌ها، خطاها و سایر اطلاعات مربوط به هر فرآیند را مشاهده کنند و مشکلات را شناسایی کنند.

۲. ارزیابی عملکرد سیستم: $Jaeger$ به تیم‌های توسعه نرم‌افزار کمک می‌کند تا عملکرد سیستم‌های خود را ارزیابی کنند و عیب‌ها و مشکلات را شناسایی کنند. این ابزار اطلاعات دقیقی از عملکرد سیستم در زمان واقعی فراهم می‌کند. ۳. بهبود عملکرد: با استفاده از $Jaeger$، توسعه‌دهندگان می‌توانند بهبودهای لازم را در سیستم‌های خود اعمال کنند و عملکرد آنها را بهبود بخشند. این ابزار به آنها کمک می‌کند تا مشکلات را سریعاً شناسایی و رفع کنند. ۴. مانیتورینگ عملکرد در محیط‌های پیچیده: $Jaeger$ به توسعه‌دهندگان کمک می‌کند تا عملکرد سیستم‌های پیچیده و توزیع‌شده را مانیتور کنند و مشکلات را در این محیط‌ها شناسایی کنند. ۵. تحلیل و بهینه‌سازی عملکرد: با تحلیل داده‌های جمع‌آوری شده توسط $Jaeger$، توسعه‌دهندگان می‌توانند عملکرد سیستم را بهبود بخشند و بهینه‌سازی‌های لازم را

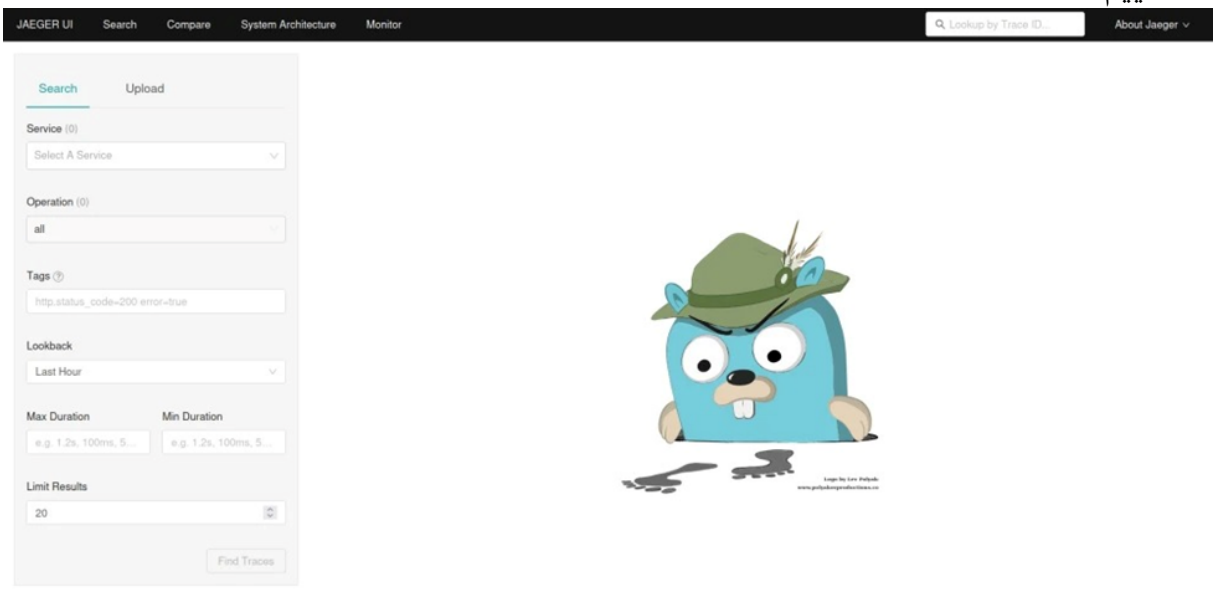
اعمال کنند.

ب)برای نصب این سیستم در کامپیوتر ابتدا فایل نصبی آن را به صورت مستقیم از سایت دانلود کرده و بعد از اجرا کردن فایل اجرایی آن ($jeager - all - in - one$) که به شکل زیر است





سیستم بر روی پورت 16686 : $http : //localhost$ بالامیاید و میتوانیم فضای اصلی آن را که به شرح زیر است مشاهده نماییم :



ج) برای اجرای کد پایتون نیازمند ماژول $opentelemetry$ میباشیم که برای نصب آن بعد از نصب پایتون روی ویندوز با دستورات زیر آنها را دانلود و نصب میکنیم :

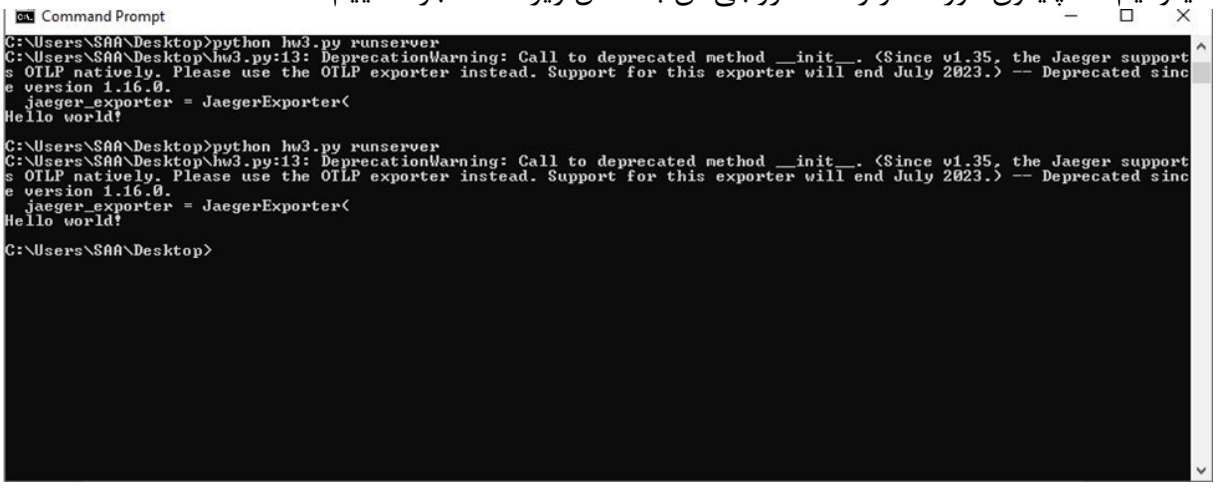
$pipinstallopentelemetry - api$

$pipinstallopentelemetry - sdk$

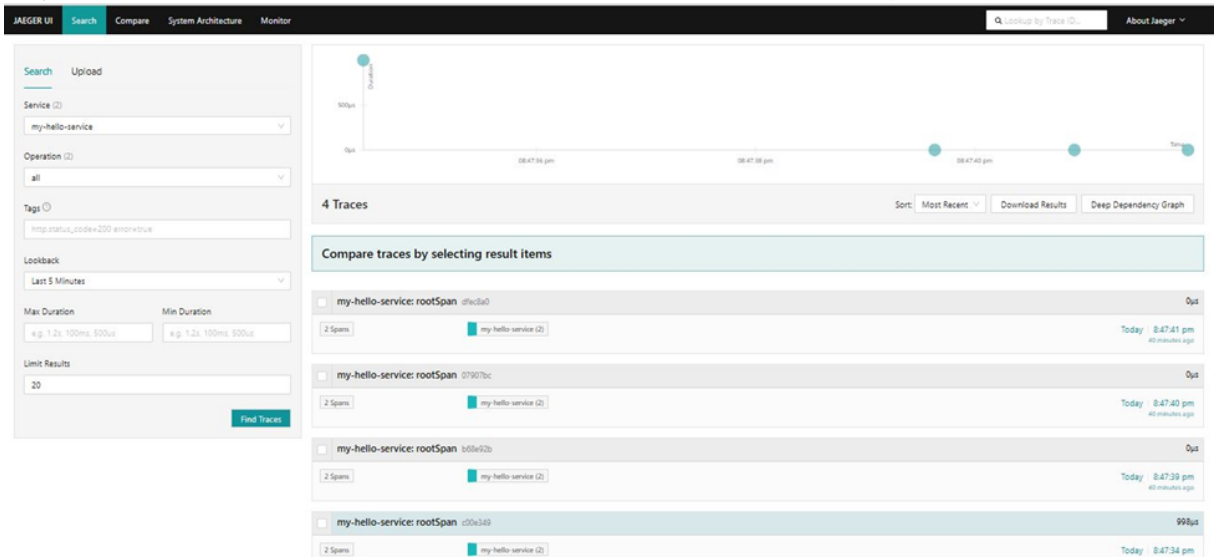
همچنین اکسپورتر آن را نیز نصب میکنیم با دستور زیر :

$pipinstallopentelemetry - exporter - jeager$

بعداز میتوانیم کد پایتون مورد نظر را که خروجی آن به شکل زیر است اجرا نماییم :



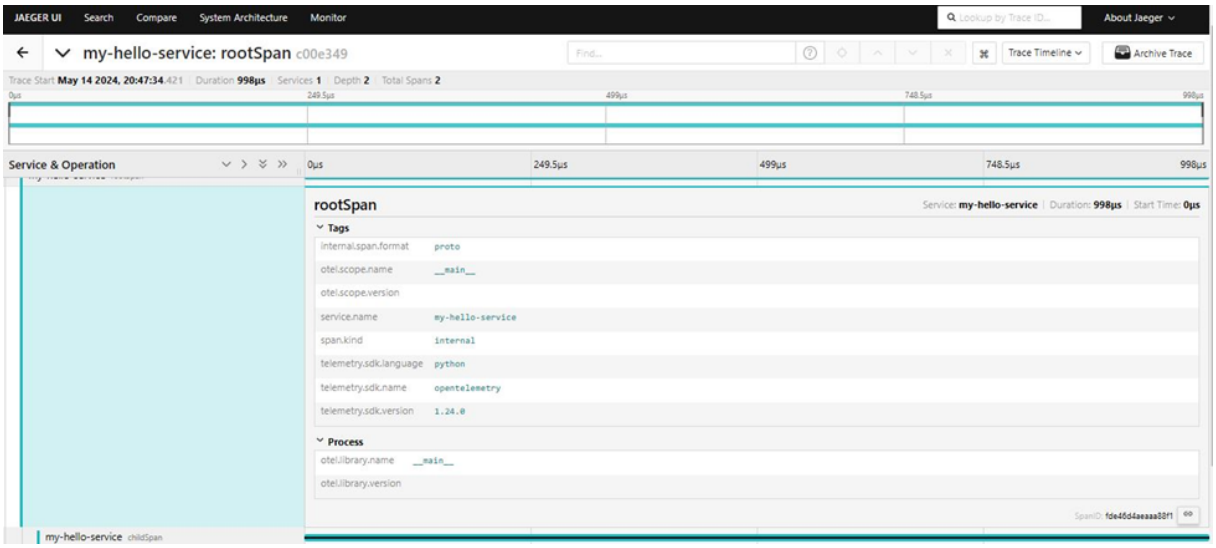
د) بعد از اینکه ما ۴ بار برنامه را ران کردیم، در صفحه اصلی سرویس را که اکسپورت پروژه مان است انتخاب میکنیم و $findtrace$ را میزنیم. نمودار اولیه که در سمت راست نمایش داده میشود اجرای هربار از برنامه را به صورت نقطه به نقطه نمایش میدهد. در پایین آن نمودار نیز $rootspan$ های هر بار اجرای برنامه را مشاهده میکنیم.



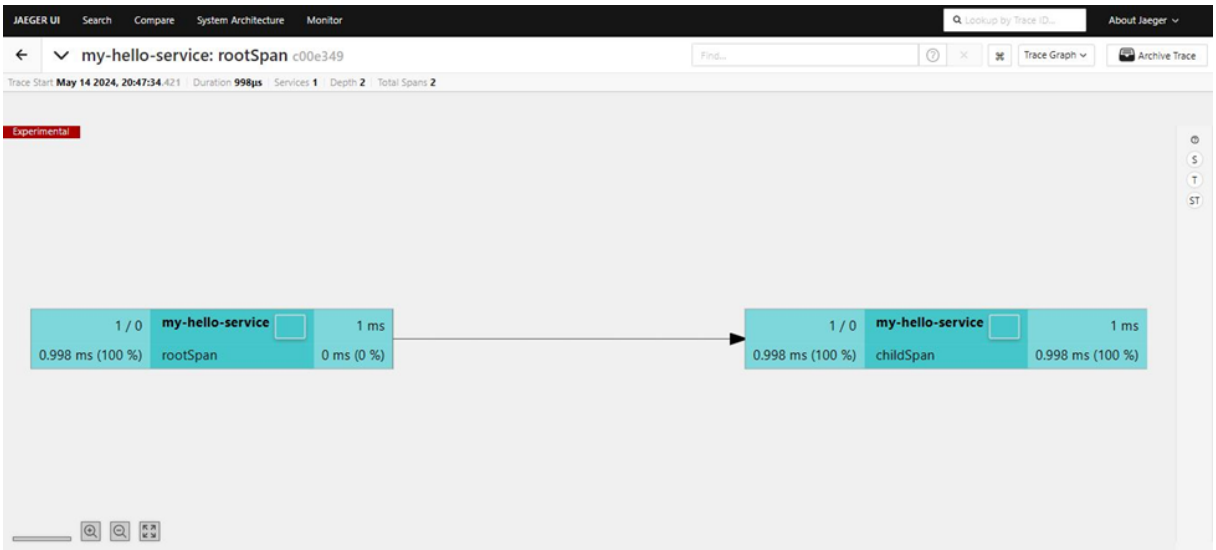
باکلید بر روی هر $rootspan$ میتوانیم جزئیات بیشتری از هر $span$ ببینیم.



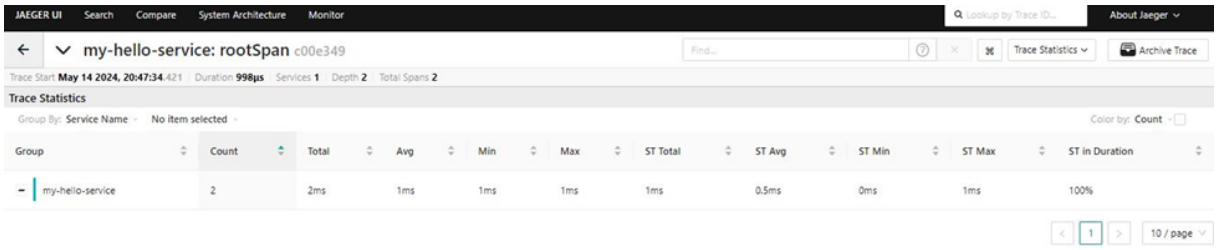
بعد از کلید روی سرویس مورد نظر، اجزاء آن همانطور که در شکل فوق مشخص است، قابل مشاهده میباشد که شامل دو اسپن $rootspan$ و $childspan$ میباشد. نمودار بالای آن نیز زمان اجرای آن را نشان میدهد.
همچنین میتوانیم با بازکردن Tag و $process$ مشخصات بیشتری از هر اسپن را ببینیم :



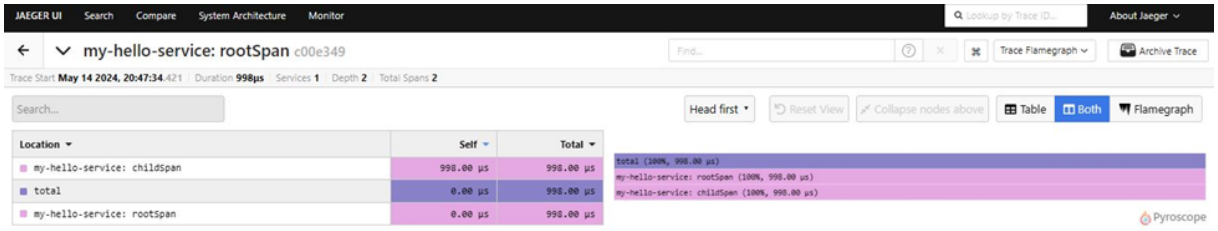
در قسمت بالای هدر میتوانیم نمودار ها و خروجی های دیگری به جز $tracetimeline$ نیز ببینیم ، برای مثال در $Tracegraph$ میبینیم که پردازه والد چقدر زمان اجرای آن بوده و به چه صورت فرایند ها صدا زده شده است :



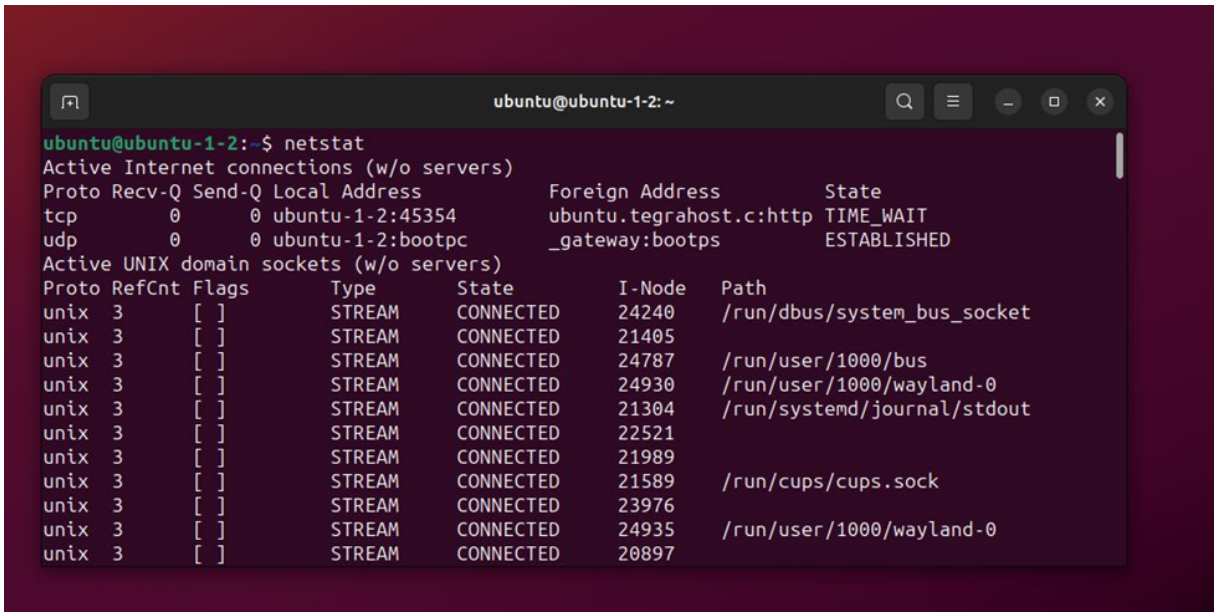
در $Tracestatistics$ نیز اطلاعات آماری سرویس ما مشخص است.



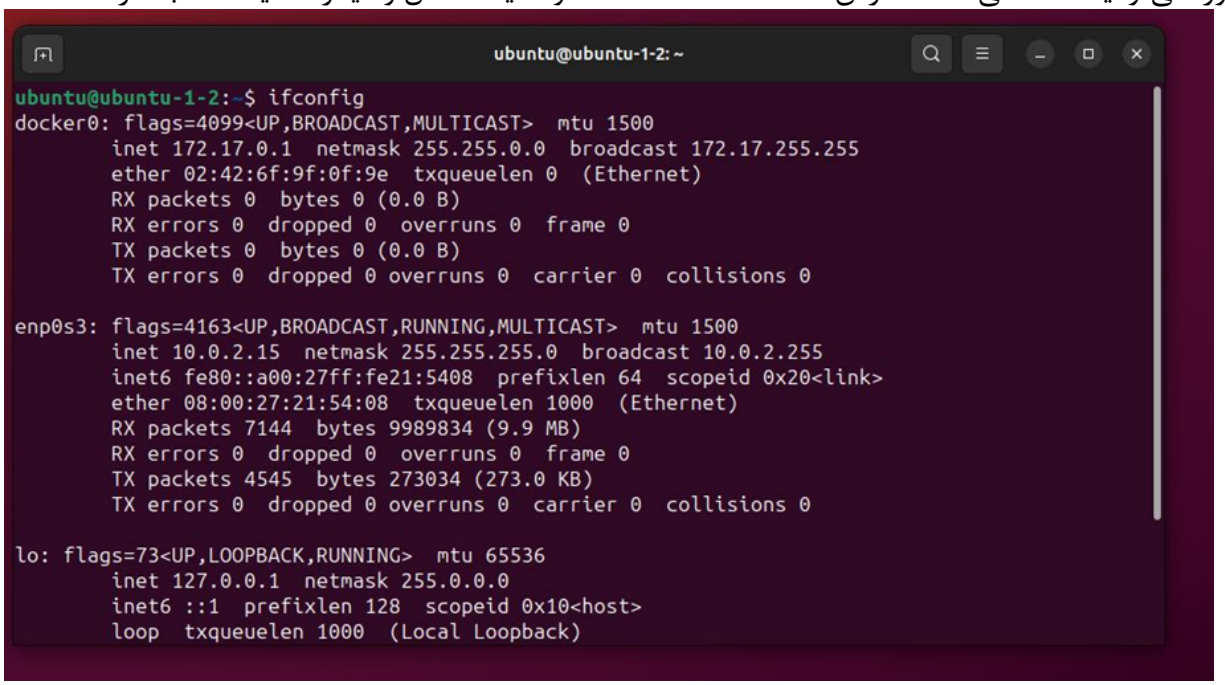
نمودار $flamegraph$ آن نیز به صورت زیر است



بخش دوم : $Netstats$ این دستور $NetworkStatistics$ (آمار شبکه) را نمایش می‌دهد. با اجرای این دستور، می‌توانید اطلاعات مربوط به اتصالات شبکه، پورت‌ها، آدرس‌های IP و سایر جزئیات شبکه را مشاهده کنید.

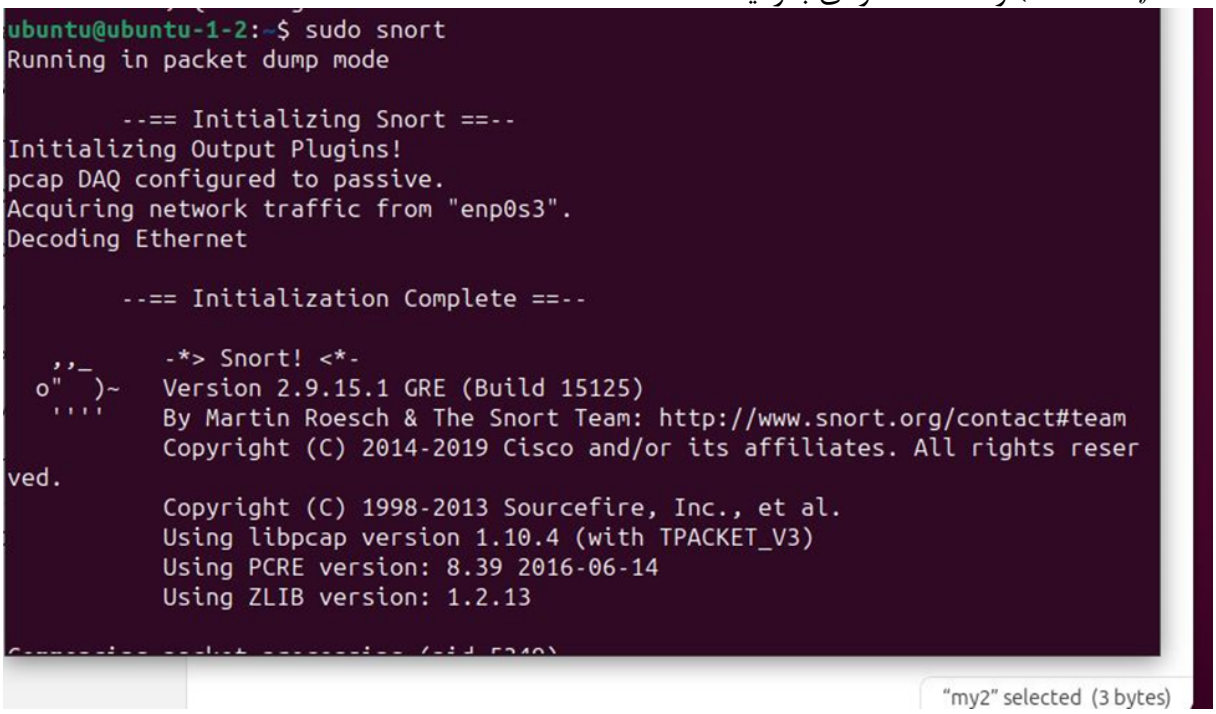


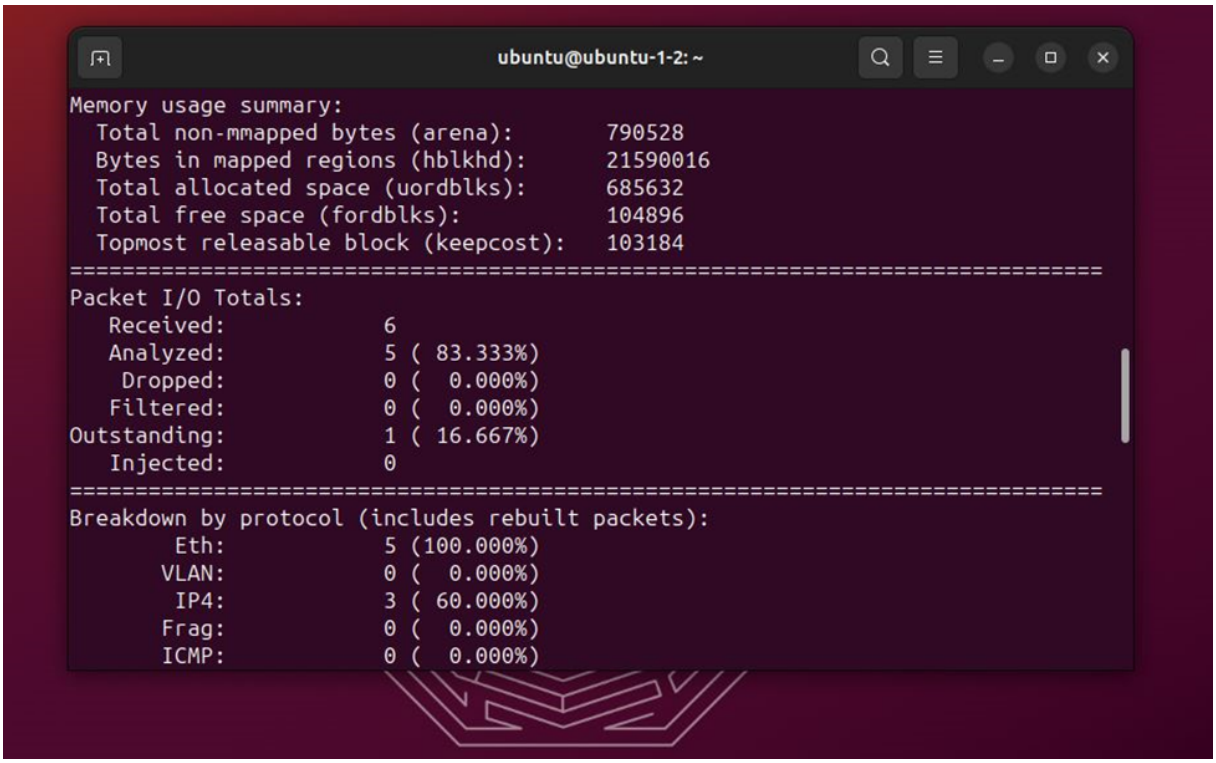
ifconfig: این دستور برای نمایش و تنظیم تنظیمات شبکه بر روی یک سیستم استفاده می‌شود. با اجرای این دستور، می‌توانید اطلاعاتی مانند آدرس IP، $MACaddress$، وضعیت اتصال و دیگر تنظیمات شبکه را مشاهده کنید.



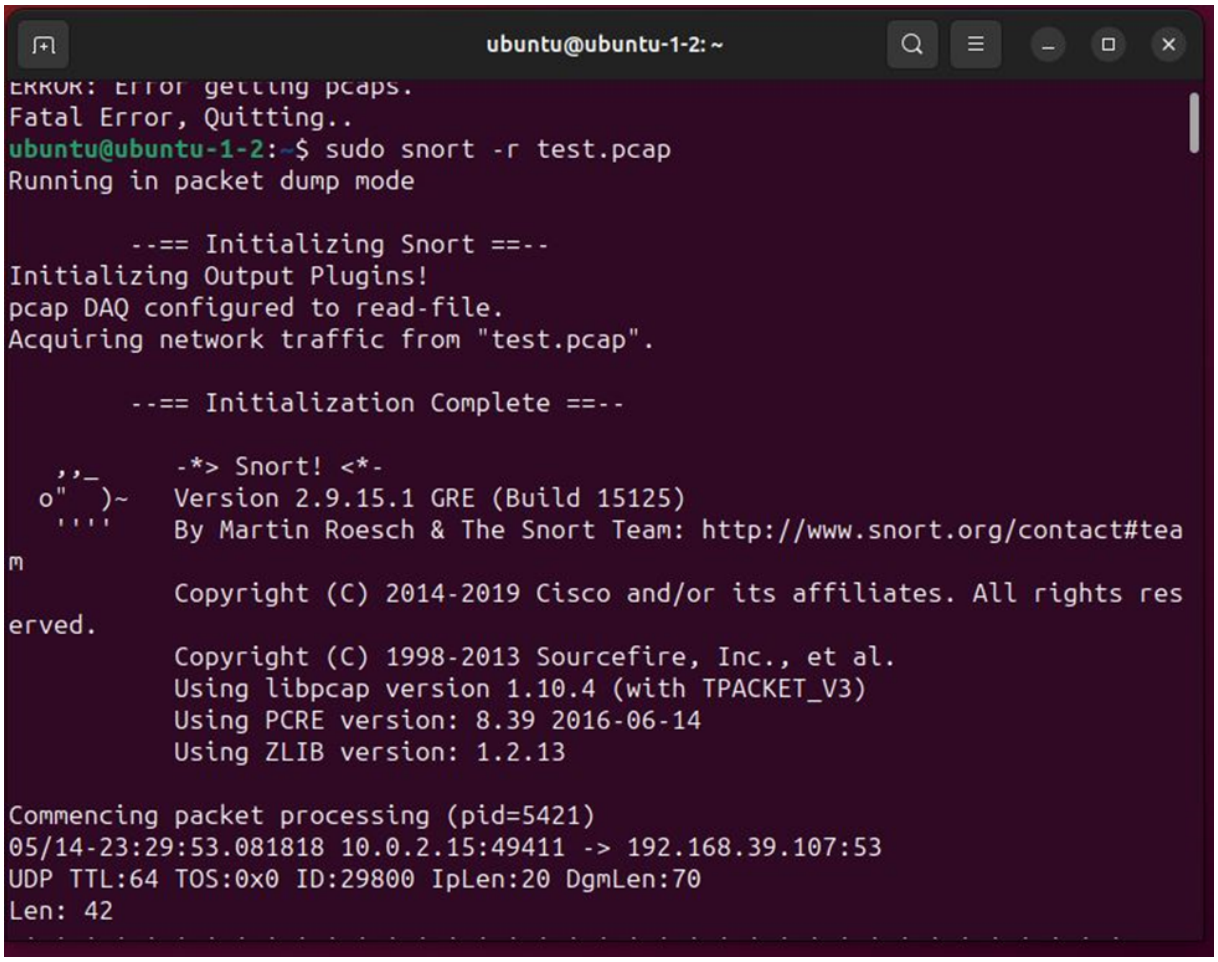
snort: $Snort$ یک سیستم مدیریت نفوذ است که برای شناسایی و پیشگیری از حملات شبکه استفاده می‌شود. با استفاده از $Snort$ می‌توانید ترافیک شبکه را مانیتور کرده و به دنبال الگوهای حملاتی مانند حملات DoS

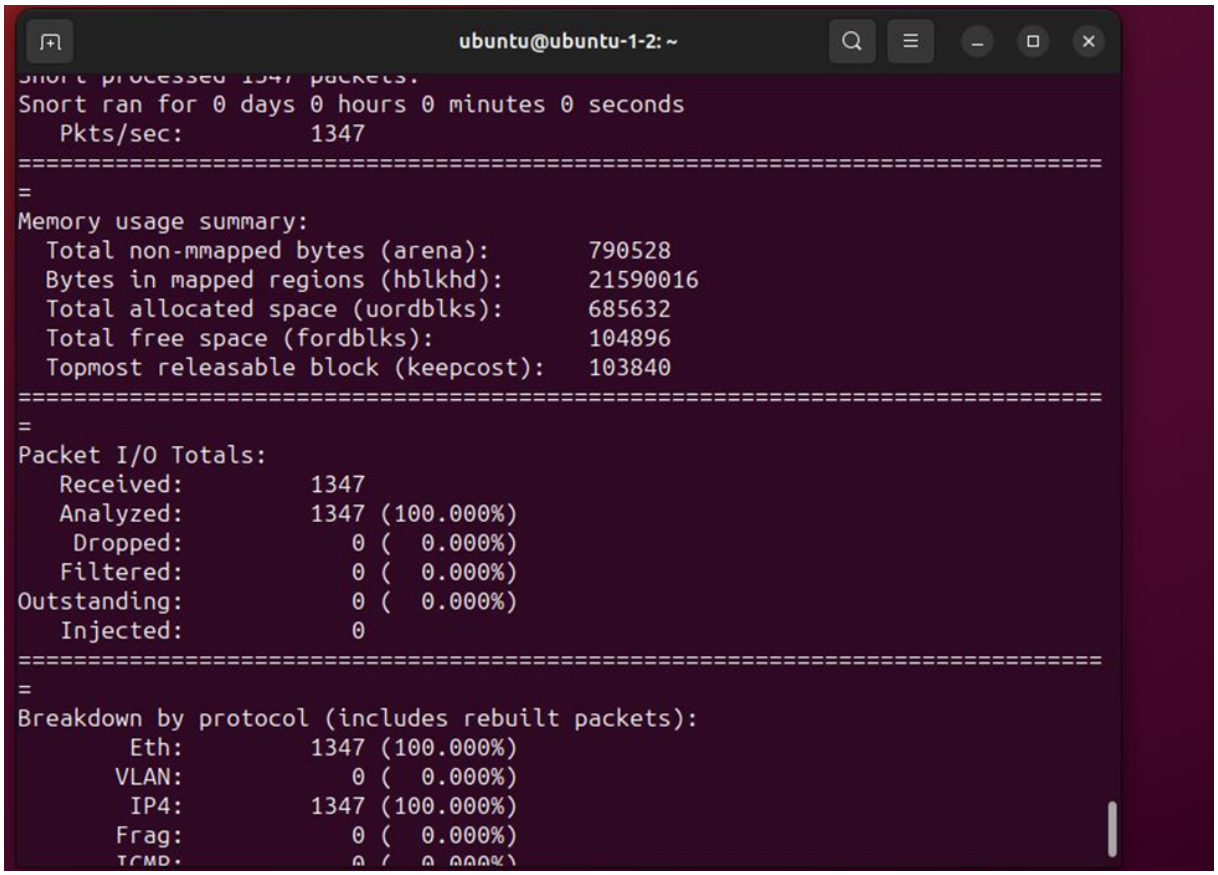
(*Denialof Service*) و حملات نفوذی بگردید.

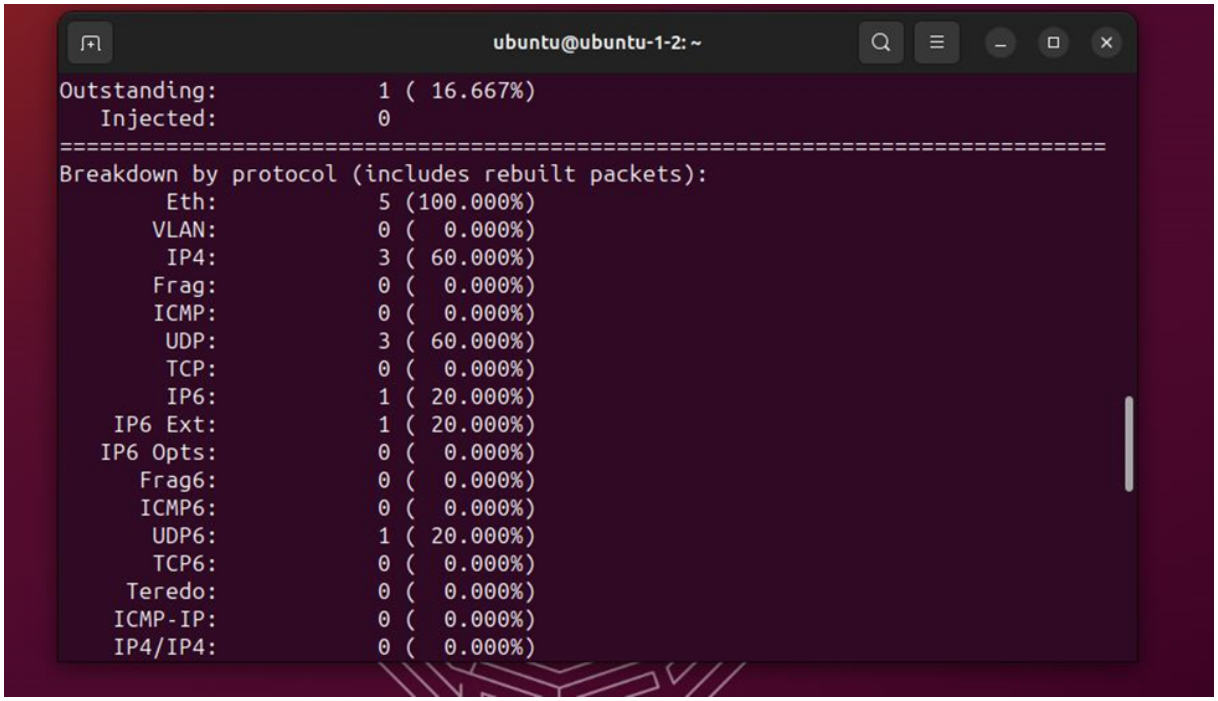




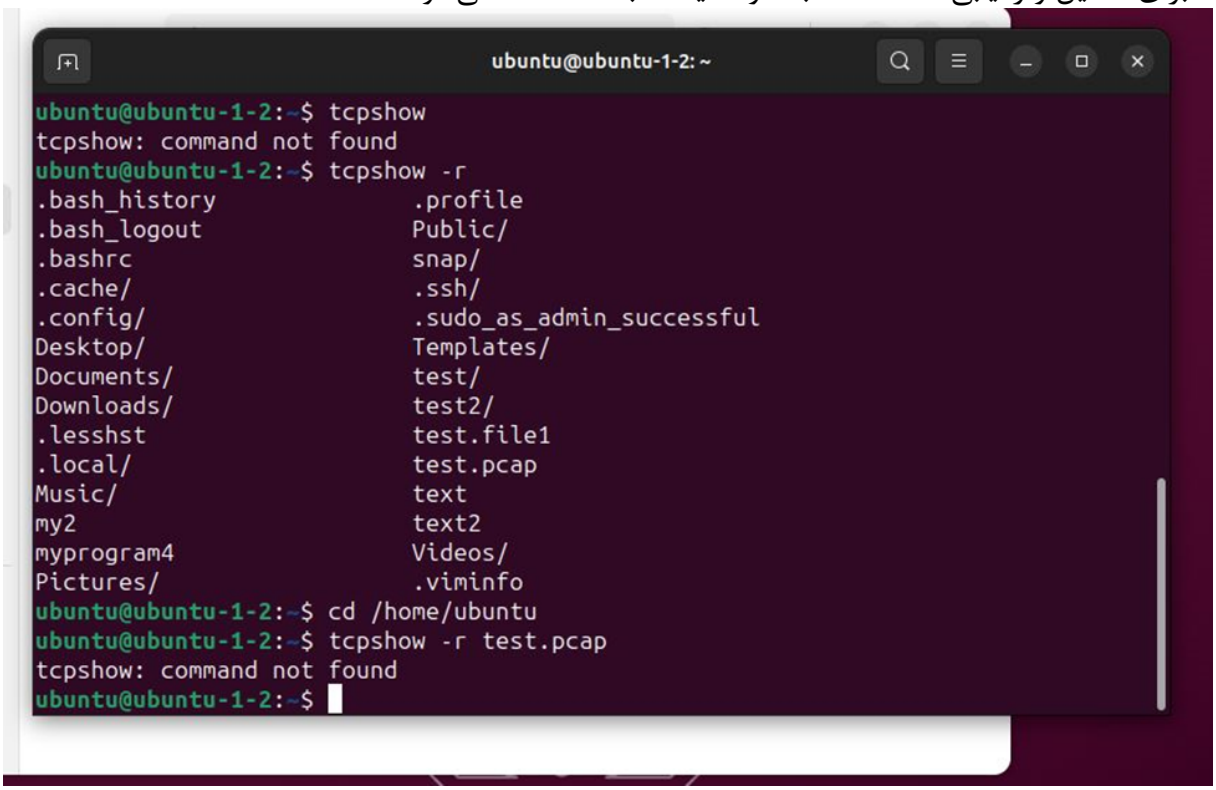
همچنین با این دستور میتوانیم فایل *pcap* را بخوانیم و تحلیل کنیم :


ubuntu@ubuntu-1-2: ~
ERROR: Error getting pcaps.
Fatal Error, Quitting..
ubuntu@ubuntu-1-2:~$ sudo snort -r test.pcap
Running in packet dump mode
--== Initializing Snort ==--
Initializing Output Plugins!
pcap DAQ configured to read-file.
Acquiring network traffic from "test.pcap".
--== Initialization Complete ==--
-*> Snort! <*-
Version 2.9.15.1 GRE (Build 15125)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2019 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.4 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.13
Commencing packet processing (pid=5421)
05/14-23:29:53.081818 10.0.2.15:49411 -> 192.168.39.107:53
UDP TTL:64 TOS:0x0 ID:29800 IpLen:20 DgmLen:70
Len: 42


ubuntu@ubuntu-1-2: ~
Snort ran for 0 days 0 hours 0 minutes 0 seconds
Pkts/sec: 1347
Memory usage summary:
Total non-mmapped bytes (arena): 790528
Bytes in mapped regions (hblkhd): 21590016
Total allocated space (uordblks): 685632
Total free space (fordblks): 104896
Topmost releasable block (keepcost): 103840
Packet I/O Totals:
Received: 1347
Analyzed: 1347 (100.000%)
Dropped: 0 (0.000%)
Filtered: 0 (0.000%)
Outstanding: 0 (0.000%)
Injected: 0
Breakdown by protocol (includes rebuilt packets):
Eth: 1347 (100.000%)
VLAN: 0 (0.000%)
IP4: 1347 (100.000%)
Frag: 0 (0.000%)

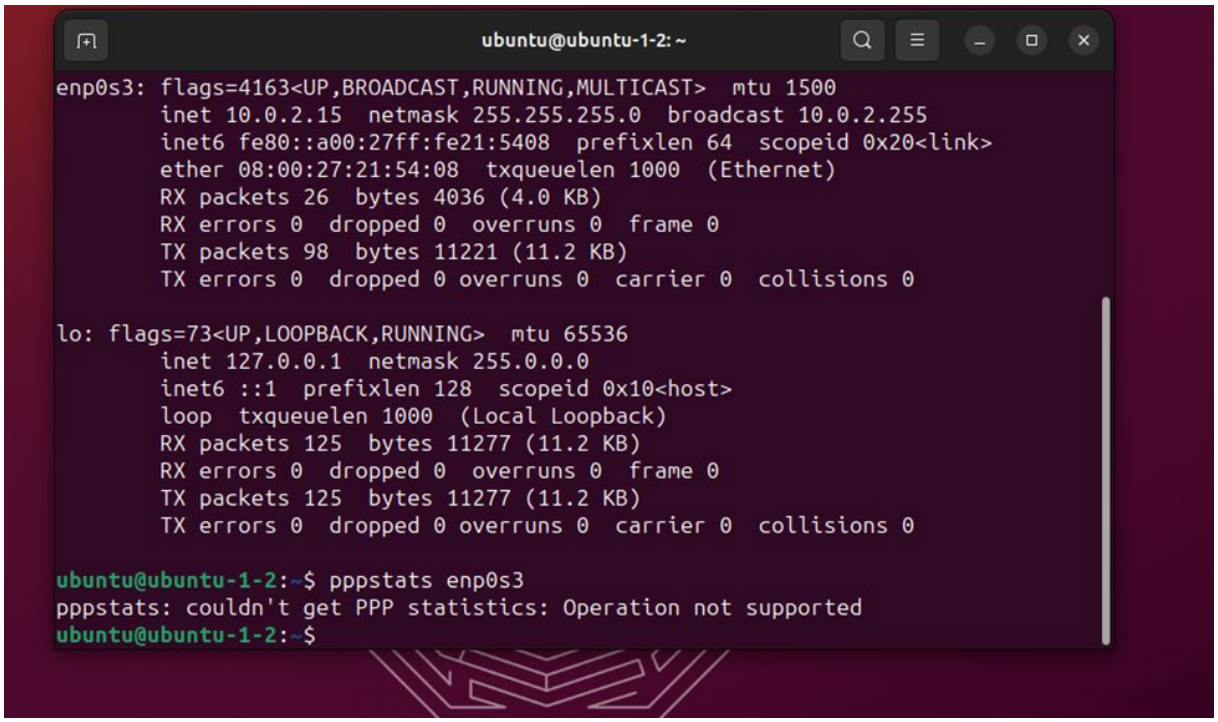

ubuntu@ubuntu-1-2: ~
Outstanding: 1 (16.667%)
Injected: 0
Breakdown by protocol (includes rebuilt packets):
Eth: 5 (100.000%)
VLAN: 0 (0.000%)
IP4: 3 (60.000%)
Frag: 0 (0.000%)
ICMP: 0 (0.000%)
UDP: 3 (60.000%)
TCP: 0 (0.000%)
IP6: 1 (20.000%)
IP6 Ext: 1 (20.000%)
IP6 Opts: 0 (0.000%)
Frag6: 0 (0.000%)
ICMP6: 0 (0.000%)
UDP6: 1 (20.000%)
TCP6: 0 (0.000%)
Teredo: 0 (0.000%)
ICMP-IP: 0 (0.000%)
IP4/IP4: 0 (0.000%)

$Tcpshow$: دستور $tcpshow$ یک ابزار خط فرمان است که بسته‌های TCP را از یک فایل $pcap$ ($PacketCapture$) مشاهده می‌کند. این دستور به کاربر امکان مشاهده اطلاعات مربوط به بسته‌های TCP ارسال و دریافت شده در یک شبکه را می‌دهد، از جمله اطلاعاتی مانند IP مبدا و مقصد، پورت مبدا و مقصد، شماره توالی بسته و غیره. این ابزار معمولا برای تحلیل و ردیابی مشکلات شبکه و امنیت شبکه استفاده می‌شود.



این دستور در واقع فایل $pcap$ را میخواند که میتواند خروجی دستور $tcpdump$ باشد. برای مثال در تصویر فوق فایل $Test.pcap$ را که خروجی دستور $tcpdump$ است را به $tcpshow$ میدهیم ولی این دستور در اینجا ساپورت نمیشود.

$Pppstats$: دستور $pppstats$ در سیستم عامل لینوکس به منظور نمایش آمار و اطلاعات مربوط به اتصال PPP ($Point-to-PointProtocol$) استفاده می‌شود. این دستور به کاربر امکان می‌دهد تا اطلاعاتی مانند تعداد بسته‌های ارسالی و دریافتی، میزان برقراری ارتباط و سرعت اتصال را بررسی کند.



همانطور که در تصویر مشخص است، این دستور حتی با مشخص کردن اینترفیس برای آن ساپورت نمیشود.